NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

III B.TECH. I SEMESTER

USER INTERFACE DESIGN USING FLUTTER LAB

Roll Number: 24471A43CP

Name of the Student: THOTAKURA BALA SUSHMA

Academic Year: 26-27

NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

Narasaraopeta Engineering College

Kotappakonda Road, Yellamanda (Post), Narasaraopet – 522601, Guntur District, AP

Approved by AICTE, New Delhi & Permanently affiliated to JNTUK, Kakinada, Code: 47,

Accredited by NBA & NAAC, RTA Approved Pollution test Centre, ISO 9001: 2008 Certified Institution. Phone: 08647-239905, Website: www.nrtec.in.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Approved by AICTE, Permanently affiliated to JNTUK, Accredited by NBA & NAAC with 'A+' Grade, Kotappakonda Road, Yellamanda (V), Narasaraopeta, Andhra Pradesh – 522601, www.nrtec.in

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE)

CERTIFICATE

ACADEMIC YEAR: 26-27

This is to certify that the bonafide record work done by THOTAKURA BALA SUSHMA bearing Roll Number 24471A43CP of III B.Tech. I Semester in the USER INTERFACE DESIGN USING FLUTTER LAB is satisfactorily completed.

Staff In-Charge **Head of the Department**

Submitted for the practical examination held on _____

External Examiner

Note: This is a computer-generated certificate, physical signature is not required

INDEX

SNo	Name of the Experiment	Program (5M)	MCQs (5M)	Execution (5M)	Total (15M)
1	Install Flutter and Dart SDK & Write a Simple Dart Program	5	5	5	15
2	Explore Flutter Widgets & Implement Layouts (Row, Column, Stack)	5	5	5	15
3	To understand and implement various layout structures in Flutter using Row, Column, and Stack widgets for building flexible UI designs.	5	5	5	15
4	To create a Flutter application that demonstrates responsiveness by adjusting layout and widget arrangement based on the screen size and orientation.	5	5	5	15
5	Stateless & Stateful Widgets – State Management using setState and Provider	5	5	5	15
6	Custom Widgets & Theming in Flutter	5	5	5	15
7	Form Design with Input Fields & Validation in Flutter	5	5	5	15
8	Adding Animations to UI Elements using Flutter's Animation Framework	5	5	5	15
9	Fetch Data from a REST API and Display in Flutter UI	5	5	5	15
10	Unit Testing UI Components & Using Flutter Debugging Tools	5	5	5	15

Marks : Average(Total) = 15.0

Student Name: THOTAKURA BALA SUSHMA **Date: Jul 21, 2026, 10:33:50 AM**

Experiment-1

AIM:

The aim of this experiment is to set up the Flutter and Dart development environment and to write simple Dart programs that illustrate the fundamental syntax and concepts of the Dart programming language.

PROBLEM STATEMENT:

PART (A):

Install Flutter and Dart SDK on your operating system (**Windows / macOS / Linux**).

Verify the installation using:

```
flutter doctor
```

and

```
dart --version
```

PART (B):

Write a simple Dart program that demonstrates:

- Declaring variables of different types (`int`, `double`, `String`, `bool`, `List`, `Map`)
- Using `var`, `final`, and `const`
- Writing a function that takes parameters and returns a value
- Using `if-else` and `for` loop / `while` loop
- Defining a class with properties and a method
- Creating an object and calling its method

INPUT:

- No user input required for basic programs.
- The Dart program can be extended to read user input using:

```
stdin.readLineSync()
```

OUTPUT:

Console output showing:

- Variable declarations
- Loop execution
- Function return values
- Class method outputs

Example Output

```
--- Variable Demo ---
Integer: 42
Double: 3.1416
String: Hello Dart
```

```
Boolean: true
List: [apple, banana, cherry]
Map: {name: Alice, age: 25}

--- final & const ---
finalVar = 100
constVar = 3.14

--- Function Demo ---
Sum of 5 and 7 = 12

--- Loop Demo ---
1 2 3 4 5

--- Class Demo ---
Person: John, Age: 30
Greeting: Hello, my name is John
```

INSTALLATION PROCEDURE (PART A):

FOR WINDOWS:

Step 1

Download the Flutter SDK from:

```
https://docs.flutter.dev/get-started/install/windows
```

Step 2

Extract the downloaded ZIP file to:

```
C:\src\flutter
```

(or any preferred location).

Step 3

Add Flutter to PATH:

```
C:\src\flutter\bin
```

Navigate:

```
System Properties
 Environment Variables
 Path
 Add Flutter bin path
```

Step 4

Open Command Prompt or PowerShell and run:

```
flutter doctor
```

Flutter checks the installation and displays missing dependencies.

Step 5

Verify Dart installation:

```
dart --version
```

FOR MACOS:

Step 1

Install Flutter using Homebrew:

```
brew install --cask flutter
```

Step 2

Alternatively, download Flutter SDK manually from Flutter's website and add it to PATH.

Step 3

Run:

```
flutter doctor
```

to complete setup.

Step 4

Verify Dart installation:

```
dart --version
```

FOR LINUX (UBUNTU):

Step 1

Download Flutter SDK:

```
cd ~/development

wget https://storage.googleapis.com/flutter_infra_release/releases/stable/linux/flutter_linu
x_3.x.x-stable.tar.xz

tar xf flutter_linux_*.tar.xz
```

Step 2

Add Flutter to PATH:

```
export PATH="$PATH:$HOME/development/flutter/bin"
```

Add the above command to:

```
~/.bashrc
```

for permanent access.

Step 3

Run:

```
flutter doctor
```

and install any missing dependencies.

Step 4

Verify Dart installation:

```
dart --version
```

ALGORITHM / PROCEDURE (PART B - DART PROGRAM):

Step 1

Create a Dart file:

```
basics.dart
```

Step 2

Write the program entry point:

```
void main() {
}
```

Step 3

Declare variables using:

```
int
double
String
bool
List
```

```
Map
```

Step 4

Demonstrate:

```
var
final
const
```

Step 5

Create a function:

```
int add(int a, int b)
```

that returns the sum.

Step 6

Use an `if-else` condition.

Step 7

Use a `for` loop or `while` loop.

Step 8

Define a class named `Person` with:

- name
- age
- greet()

Step 9

Create an object of `Person`.

Step 10

Run the program:

```
dart basics.dart
```

EXAMPLES:

EXAMPLE 1 - HELLO WORLD IN DART:

```
void main() {
  print('Hello, Dart!');
}
```

Output

```
Hello, Dart!
```

EXAMPLE 2 - FUNCTION WITH RETURN VALUE:

```
int multiply(int x, int y) => x * y;

void main() {
  print(multiply(4, 5));
}
```

Output

```
20
```

EXAMPLE 3 - VARIABLE DECLARATIONS:

```
void main() {
  int age = 25;
  double pi = 3.1416;
  String name = "Alice";
  bool isStudent = true;

  List<String> fruits = [
    "apple",
    "banana",
    "cherry"
  ];

  Map<String, dynamic> person = {
    "name": "Alice",
    "age": 25
  };

  print(age);
  print(pi);
  print(name);
  print(isStudent);
  print(fruits);
  print(person);
}
```

EXAMPLE 4 - USING VAR, FINAL, AND CONST:

```
void main() {
  var city = "London";

  final int finalVar = 100;
```

```
  const double constVar = 3.14;

  print(city);
  print(finalVar);
  print(constVar);
}
```

EXAMPLE 5 - IF-ELSE STATEMENT:

```
void main() {
  int marks = 75;

  if (marks >= 50) {
    print("Pass");
  } else {
    print("Fail");
  }
}
```

Output

```
Pass
```

EXAMPLE 6 - FOR LOOP:

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print(i);
  }
}
```

Output

```
1
2
3
4
5
```

EXAMPLE 7 - CLASS AND OBJECT:

```
class Person {
  String name;
  int age;

  Person(this.name, this.age);
```

```
  void greet() {
    print("Hello, my name is $name");
  }
}

void main() {
  Person p = Person("John", 30);

  print("Person: ${p.name}, Age: ${p.age}");

  p.greet();
}
```

Output

```
Person: John, Age: 30
Hello, my name is John
```

CONSTRAINTS:

- Flutter and Dart SDK require approximately **2.5 GB** of free disk space.
- Windows users should use **PowerShell 5.0+** or Command Prompt.
- macOS users may install **Xcode** for iOS development (optional for Dart-only programs).
- Android Studio or VS Code with Flutter extensions are optional but recommended.
- Every Dart program must contain:

```
void main()
```

- Use **Null Safety** (Dart 2.12+) whenever possible.
- Avoid explicit `null` values unless using nullable types:

```
String? name;
```

- Programs should compile and run using:

```
dart basics.dart
```

or through Flutter-supported IDEs.

STUDENT SUBMISSION / CODE:

File Path: experiment1.dart

```
import 'dart:io';
void main(){
  var firstName = "John";
```

```
  var lastName = "Doe";
  int num1 = 10;
  int num2 = 3;
  int sum = num1 + num2;
  int diff = num1 - num2;
  int mul = num1 * num2;
  double div = num1 / num2;
  print("Enter number:");
  int? number = int.parse(stdin.readLineSync()!);
  print("The entered number is ${number}");
  print("The sum is $sum");
  print("The diff is $diff");
  print("The mul is $mul");
  print("The div is $div");
  print("Full name is $firstName $lastName");
}

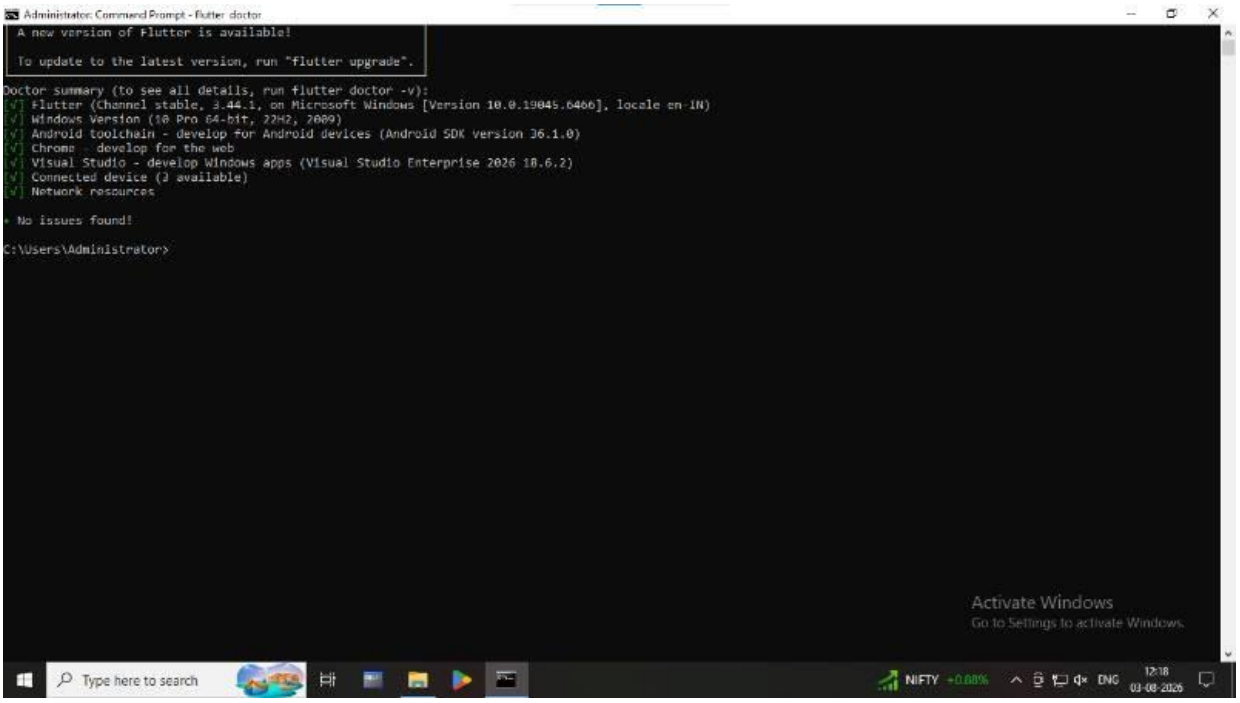
OUTPUT:
Enter number:
10
The entered number is 10
The sum is 13
The diff is 7
The mul is 30
The div is 3.3333333333333335
Full name is John Doe

Process finished with exit code 0
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: Allabhakshu4348
Repo Name: simpledartprogram
Default Branch: main
Fetched Files:
 - experiment1.dart
```

Output Image: output1.png



Student Name: THOTAKURA BALA SUSHMA **Date: Jul 29, 2026, 11:47:16 AM**

Experiment-2

AIM:

The aim of this experiment is to explore fundamental Flutter widgets and to implement different layout structures using **Row**, **Column**, and **Stack** widgets to arrange UI components.

PROBLEM STATEMENT:

PART (A): BASIC FLUTTER WIDGETS:

Write a Flutter program that demonstrates:

- `Text` with different styles (font size, color, font weight)
- `Image` (network image or local asset) displayed inside a `Container`
- `Container` with:
  - Padding
  - Margin
  - Decoration (`BoxDecoration`)
  - Border
  - Background color or gradient

PART (B): LAYOUT WIDGETS:

Write a Flutter program that demonstrates:

Row

Arrange three icons or text widgets horizontally with spacing.

Column

Arrange:

- Heading
- Image
- Description

vertically.

Stack

Display overlapping widgets such as:

- Text over an image
- Multiple containers layered together

Combine all demonstrations into a single Flutter application using:

- Drawer
- TabBar

or display them sequentially on one screen.

INPUT:

- No direct user input required.
- Images may be loaded from:
  - Network URLs
  - Local assets

OUTPUT:

A Flutter application showing:

- Styled text widgets
- Image inside a decorated container
- Horizontal row layout
- Vertical column layout
- Stack with overlapping content

Example Visual Layout

```
Flutter Layouts

Text & Container Demo

[Styled Text]

[Decorated Container]
[       Image        ]

Row Demo



Column Demo

Title
[ Image ]
Description

Stack Demo

[ Background Image ]
[ Overlay Text ]
```

ALGORITHM / PROCEDURE:

Step 1

Create a Flutter project:

```
flutter create widget_layout_demo
cd widget_layout_demo
```

Step 2

Replace the contents of:

```
lib/main.dart
```

with the Flutter implementation.

Step 3

For local images, create:

```
assets/images/
```

and place image files inside.

Update:

```
pubspec.yaml
```

```
flutter:
  assets:
    - assets/images/
```

Step 4

Implement Basic Widgets

Text Widget

Use different `TextStyle` properties:

- fontSize
- color
- fontWeight

Image Widget

Use:

```
Image.network()
```

or

```
Image.asset()
```

inside a `Container`.

Container Widget

Apply:

```
padding
margin
decoration
```

using `BoxDecoration`.

Step 5

Implement Row Layout

Use:

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
)
```

Children may include:

```
Icon(Icons.star)
Icon(Icons.favorite)
Icon(Icons.thumb_up)
```

Step 6

Implement Column Layout

Create a vertical arrangement containing:

```
Heading
Image
Description
```

using:

```
Column()
```

Step 7

Implement Stack Layout

Use:

```
Stack()
```

with:

- Background image/container
- Positioned text label

Step 8

Wrap all sections inside:

```
ListView()
```

for scrolling.

Use:

```
Scaffold
```

with an:

```
AppBar
```

Step 9

Run the application:

```
flutter run
```

EXAMPLES:

EXAMPLE 1 - BASIC CONTAINER WITH TEXT:

```
Container(
  padding: EdgeInsets.all(16),
  margin: EdgeInsets.symmetric(
    vertical: 8,
    horizontal: 16,
  ),
  decoration: BoxDecoration(
    color: Colors.blue[100],
    borderRadius: BorderRadius.circular(12),
  ),
  child: Text(
    'Hello, Flutter!',
  ),
)
```

Output

```
A blue rounded container
containing the text:

Hello, Flutter!
```

EXAMPLE 2 - STYLED TEXT:

```
Text(
  'Flutter Widgets',
  style: TextStyle(
    fontSize: 24,
    color: Colors.blue,
    fontWeight: FontWeight.bold,
  ),
)
```

Output

```
Flutter Widgets
```

Displayed in:

- Blue color
- Large font
- Bold style

EXAMPLE 3 - IMAGE INSIDE CONTAINER:

```
Container(
  padding: EdgeInsets.all(10),
  decoration: BoxDecoration(
    border: Border.all(color: Colors.black),
  ),
  child: Image.network(
    'https://picsum.photos/300/200',
  ),
)
```

Output

```
Bordered container
containing an image
```

EXAMPLE 4 - ROW WITH ICONS:

```
Row(
  mainAxisAlignment:
      MainAxisAlignment.center,
  children: [
    Icon(Icons.home, size: 40),
    SizedBox(width: 20),
    Icon(Icons.person, size: 40),
    SizedBox(width: 20),
    Icon(Icons.settings, size: 40),
  ],
)
```

Output

displayed horizontally.

EXAMPLE 5 - COLUMN LAYOUT:

```
Column(
  crossAxisAlignment:
      CrossAxisAlignment.center,
  children: [
    Text('Nature'),
    Image.network(
      'https://picsum.photos/250/150',
    ),
    Text(
```

```
      'A beautiful landscape image.',
    ),
  ],
)
```

Output

```
Nature

[Image]

A beautiful landscape image.
```

displayed vertically.

EXAMPLE 6 - STACK WITH OVERLAY TEXT:

```
Stack(
  children: [
    Image.network(
      'https://picsum.photos/300/200',
    ),
    Positioned(
      bottom: 10,
      left: 10,
      child: Container(
        color: Colors.black54,
        child: Text(
          'Overlay Text',
          style: TextStyle(
            color: Colors.white,
          ),
        ),
      ),
    ),
  ],
)
```

Output

```
Background Image
      +
Overlay Text
```

displayed on top of the image.

EXAMPLE 7 - COMPLETE LAYOUT STRUCTURE:

```
AppBar
```

```
ListView

 Text Demo
 Container Demo
 Row Demo
 Column Demo
 Stack Demo
```

CONSTRAINTS:

- Flutter SDK must be installed and configured correctly.
- The project should compile without errors.

For Local Images

Assets must be declared in:

```
pubspec.yaml
```

Example:

```
flutter:
  assets:
    - assets/images/
```

For Network Images

Internet connectivity is required.

No additional permissions are needed for standard Flutter applications.

Widget Requirements

- Use `StatelessWidget` for simplicity.
- Avoid unnecessary state management.
- Wrap large content inside `ListView` to prevent overflow.

Flutter Version

Compatible with:

```
Flutter 3.x+
Dart 3.x+
```

Layout Guidelines

- Use `Row` for horizontal layouts.
- Use `Column` for vertical layouts.
- Use `Stack` for overlapping widgets.
- Use `Container` for styling and spacing.
- Use `SizedBox` to create spacing between widgets.

Run Command

```
flutter run
```

The application should successfully launch on:

- Android Emulator
- iOS Simulator
- Physical Device

STUDENT SUBMISSION / CODE:

File Path: experiment2 (4).dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: MyLayout(),
      ),
    );
  }
}

class MyLayout extends StatelessWidget {
  const MyLayout({super.key});

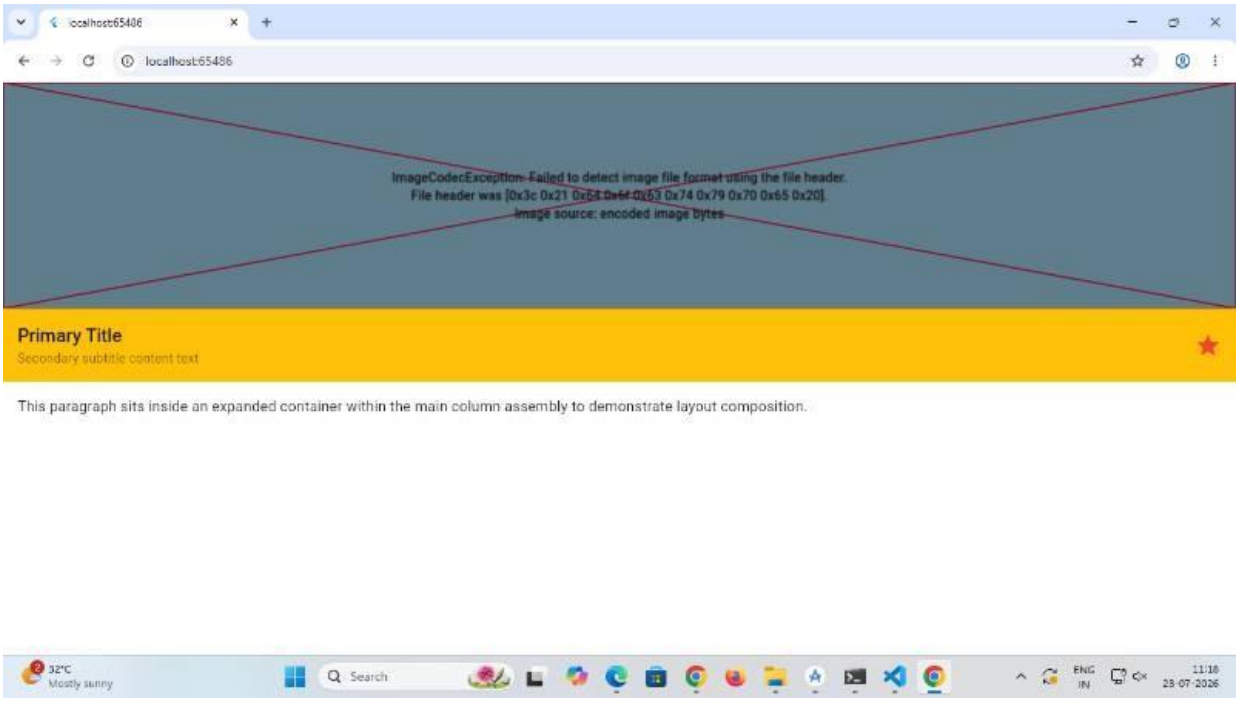
  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.start,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Container(
            height: 250,
            color: Colors.blueGrey,
            child: Image.network(
              'https://picsum.photos',
              fit: BoxFit.cover,
            ),
          ),
          Container(
            padding: const EdgeInsets.all(16),
            color: Colors.amber,
```

```
            child: const Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      'Primary Title',
                      style: TextStyle(
                        fontSize: 20,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                    Text(
                      'Secondary subtitle content text',
                      style: TextStyle(
                        fontSize: 14,
                        color: Colors.black54,
                      ),
                    ),
                  ],
                ),
                Icon(
                  Icons.star,
                  color: Colors.red,
                  size: 30,
                ),
              ],
            ),
          ),
          Expanded(
            child: Container(
              padding: const EdgeInsets.all(16),
              color: Colors.white,
              child: const Text(
                'This paragraph sits inside an expanded container within the main column
assembly to demonstrate layout composition.',
                style: TextStyle(
                  fontSize: 16,
                  height: 1.5,
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutterlab_exp-2
Default Branch: main
Fetched Files:
 - experiment2 (4).dart
```

Output Image: experiment 2 output.png



Student Name: THOTAKURA BALA SUSHMA **Date: Aug 7, 2026, 10:02:43 PM**

Experiment-3

AIM:

The aim of this experiment is to design and implement a responsive UI in Flutter that adapts to different screen sizes and orientations. The experiment has two parts:

(a) Design a responsive UI that adapts to different screen sizes using `MediaQuery`, `Flexible`, and `Expanded`.

(b) Implement media queries and breakpoints to switch between different layouts (mobile, tablet, desktop) and handle orientation changes.

PROBLEM STATEMENT:

Create a Flutter application that displays a dashboard with a header, a content area, and a footer. The layout must:

1. Use **MediaQuery** to detect the screen width and orientation.
2. Define **breakpoints**:
   - Mobile: width < 600 -> single column layout (stacked widgets)
   - Tablet: 600 "dwidth < 1024 -> two-column layout
   - Desktop: width "e1024 -> three-column layout with a side navigation drawer (or rail)
3. Respond to orientation changes:
   - On mobile, switch from row-based to column-based layout when rotating to portrait mode.
4. Use **LayoutBuilder** to make inner components adapt to the space they receive.
5. Use **OrientationBuilder** to rebuild when orientation changes.

Write the complete Flutter code (`main.dart`) that meets these requirements.

INPUT:

- No user input - the app uses the device's screen size and orientation automatically.

OUTPUT:

- A responsive UI that changes layout based on screen width and orientation.
- Visual feedback showing the current screen size category (e.g., "Mobile Portrait", "Tablet Landscape") and the active layout.

Example Output (Visual Description)

- On a small phone (width < 600): all widgets stack vertically. Header, content cards, and footer appear one below another.
- On a tablet (width 600-1024): content area uses two cards side-by-side, header and footer span full width.
- On a desktop (width > 1024): a side rail (or drawer) appears on the left, and the main content uses three columns.

ALGORITHM:

Step-1: Create a new Flutter project:

```
flutter create responsive_ui_demo
```

Step-2: In `lib/main.dart`, write a `MaterialApp` with a `home` widget that obtains `MediaQueryData` and `Orientation`.

Step-3: Define a top-level function `getScreenCategory(width)` that returns `"mobile"`, `"tablet"`, or `"desktop"`.

Step-4: Build a `Scaffold`. Use `OrientationBuilder` to rebuild when orientation changes.

Step-5: For the body, use `LayoutBuilder` to access `BoxConstraints`.

Step-6: Inside `LayoutBuilder`, conditionally return different widget trees based on width breakpoints.

Step-7: For mobile (width < 600), use a `Column` with children stacked vertically.

Step-8: For tablet (600-1024), use a `Row` or `GridView` with two columns for content.

Step-9: For desktop ("e 1 us0e a2`Row`4 wi)th a, side navigation `Drawer` or a `NavigationRail`.

Step-10: Use `Expanded` and `Flexible` to distribute space proportionally.

Step-11: Test the app on an emulator with different screen sizes and orientations.

EXAMPLES:

Example 1 - Using MediaQuery to Get Screen Width

```
double width = MediaQuery.of(context).size.width;

String category = width < 600
    ? "Mobile"
    : (width < 1024 ? "Tablet" : "Desktop");
```

Example 2 - OrientationBuilder

```
OrientationBuilder(
  builder: (context, orientation) {
    return orientation == Orientation.portrait
        ? Column(children: [...])
        : Row(children: [...]);
  },
)
```

Example 3 - LayoutBuilder for Adaptive Padding

```
LayoutBuilder(
  builder: (context, constraints) {
    double padding = constraints.maxWidth < 600 ? 8.0 : 16.0;

    return Padding(
      padding: EdgeInsets.all(padding),
      child: ...
    );
  },
)
```

CONSTRAINTS:

- Flutter SDK must be installed and configured.
- Run on an emulator or physical device with different screen sizes (e.g., Nexus 5, Pixel C, desktop).
- Avoid hard-coding fixed widths that may overflow on smaller screens.
- Use proper `Expanded` and `Flexible` instead of `SizedBox` with fixed sizes when inside flexible layouts.
- Ensure the application adapts smoothly to orientation changes.
- Layouts should remain readable and usable across all supported screen sizes.
- Test mobile, tablet, and desktop breakpoints before submission.

STUDENT SUBMISSION / CODE:

File Path: experiment3.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Layout Demo',
      home: const LayoutDemo(),
    );
  }
}

class LayoutDemo extends StatelessWidget {
  const LayoutDemo({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Row, Column & Stack Demo'),
        centerTitle: true,
      ),

      body: SingleChildScrollView(
        child: Padding(
          padding: const EdgeInsets.all(16.0),

          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [

// Column Example
              const Text(
                "Column Widget",
                style: TextStyle(
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                ),
              ),

              const SizedBox(height: 10),

              Container(
                height: 60,
                color: Colors.blue,
```

```
                child: const Center(
                  child: Text(
                    "Item 1",
                    style: TextStyle(color: Colors.white),
                  ),
                ),
              ),

              const SizedBox(height: 10),

              Container(
                height: 60,
                color: Colors.green,
                child: const Center(
                  child: Text(
                    "Item 2",
                    style: TextStyle(color: Colors.white),
                  ),
                ),
              ),

              const SizedBox(height: 30),

// Row Example
              const Text(
                "Row Widget",
                style: TextStyle(
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                ),
              ),

              const SizedBox(height: 10),

              Row(
                children: [

                  Expanded(
                    child: Container(
                      height: 80,
                      color: Colors.red,
                      child: const Center(
                        child: Text(
                          "Box 1",
                          style: TextStyle(color: Colors.white),
                        ),
                      ),
                    ),
                  ),

                  const SizedBox(width: 10),

                  Expanded(
                    child: Container(
                      height: 80,
                      color: Colors.orange,
```

```
                    child: const Center(
                      child: Text(
                        "Box 2",
                        style: TextStyle(color: Colors.white),
                      ),
                    ),
                  ),
                ),
              ],
            ),

            const SizedBox(height: 30),

// Stack Example
            const Text(
              "Stack Widget",
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),

            const SizedBox(height: 10),

            Center(
              child: Stack(
                alignment: Alignment.center,
                children: [

                  Container(
                    width: 220,
                    height: 220,
                    color: Colors.blue.shade200,
                  ),

                  Container(
                    width: 150,
                    height: 150,
                    color: Colors.green.shade400,
                  ),

                  Container(
                    width: 80,
                    height: 80,
                    color: Colors.red,
                  ),

                  const Text(
                    "Stack",
                    style: TextStyle(
                      color: Colors.white,
                      fontWeight: FontWeight.bold,
                      fontSize: 20,
                    ),
                  ),
                ],
```

```
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutterlab_exp-3
Default Branch: main
Fetched Files:
 - experiment3.dart
```

Output Image: experiment 3 output.png



Student Name: THOTAKURA BALA SUSHMA **Date: Aug 25, 2026, 2:37:52 PM**

Experiment-4

AIM:

The aim of this experiment is to create a Flutter application that demonstrates responsiveness by adjusting layout and widget arrangement based on the screen size and orientation. The app will present a set of content cards that rearrange themselves into different patterns depending on the device category and orientation.

PROBLEM STATEMENT:

Develop a Flutter application that shows a gallery of items (e.g., products, articles, or photos). The app must:

1. Detect screen width using `MediaQuery` and classify it as Mobile, Tablet, or Desktop.
2. Adapt the layout:
   - Mobile (width < 600): Single-column list/column of cards.
   - Tablet (600 "dwidth < 1024): Two-column grid of cards.
   - Desktop (width "e1024): Three-column grid or a row with a side navigation rail.
3. Respond to orientation changes:
   - On mobile, when rotated to landscape, change from a vertical list to a horizontal scrolling list or a two-column grid.
   - On tablet, adjust padding and margins when rotated.
4. Use `LayoutBuilder` at the top-level to decide the layout strategy, and `OrientationBuilder` inside to handle orientation-specific fine-tuning.
5. Display a footer that shows the current screen width, device category, and orientation for debugging/demonstration.

INPUT:

- No explicit user input; the application automatically uses the device's current screen dimensions and orientation.

OUTPUT:

- A visually distinct layout for Mobile, Tablet, and Desktop.
- When rotating the device, the layout updates immediately.
- A footer with information about the detected category and orientation.

Example Visual Description

- **Mobile Portrait:** Cards stacked vertically, each filling the width.
- **Mobile Landscape:** Cards displayed side-by-side using a `Row` with horizontal scrolling.
- **Tablet Portrait:** Two columns of cards with moderate spacing.
- **Tablet Landscape:** Two columns but with wider spacing and larger text.
- **Desktop:** A three-column grid (or a row with three expanded cards) and a side navigation rail.

ALGORITHM:

Step-1: Create a new Flutter project:

```
flutter create responsive_orientation_demo
```

Step-2: Replace `lib/main.dart` with the complete code.

Step-3: Define a list of data (e.g., 6 dummy items with title, description, icon).

Step-4: In the `build()` method:

- Use `MediaQuery.of(context).size.width` to get the screen width.
- Use `MediaQuery.of(context).orientation` to get the orientation.
- Use `LayoutBuilder` to obtain `BoxConstraints` and decide the base layout.

Step-5: Implement three separate layout builders:

- `_buildMobileLayout(context, orientation)`
- `_buildTabletLayout(context, orientation)`

- `_buildDesktopLayout(context)`

Step-6: For mobile landscape, use a `SingleChildScrollView` with a `Row` of cards or a horizontal `ListView`.

Step-7: For tablet, adjust padding and margins using `OrientationBuilder`.

Step-8: Add a `BottomAppBar` or footer widget displaying:

- Device category
- Screen width
- Orientation

Step-9: Test the application on different emulator sizes and orientations.

EXAMPLES:

Example: Using OrientationBuilder Inside Mobile Layout

```
OrientationBuilder(
  builder: (context, orientation) {
    if (orientation == Orientation.portrait) {
      return Column(children: cardList);
    } else {
      return SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        child: Row(children: cardList),
      );
    }
  },
)
```

Example: Getting Device Category Using MediaQuery

```
double width = MediaQuery.of(context).size.width;

String category = width < 600
    ? 'Mobile'
    : (width < 1024 ? 'Tablet' : 'Desktop');
```

CONSTRAINTS:

- The application must run without errors on Android, iOS, and Web/Desktop.
- Avoid hard-coding widths that may overflow on smaller screens.
- Use `Expanded` or `Flexible` inside `Row` widgets to prevent overflow.
- For horizontal scrolling on mobile landscape, ensure content does not clip.
- The footer information must update automatically when the screen size or orientation changes.
- Use `LayoutBuilder`, `MediaQuery`, and `OrientationBuilder` appropriately for responsive design.

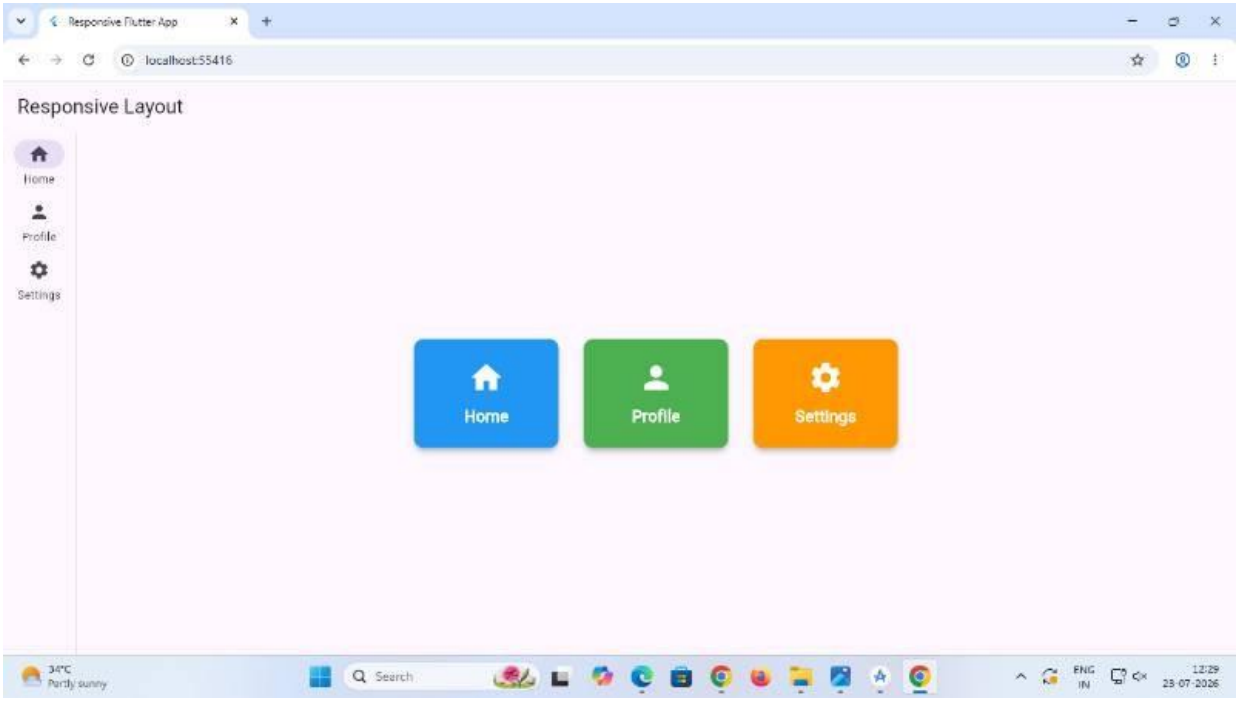
STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutter-exp---4.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutter-exp---4
Default Branch: main
Fetched Files:
```

Output Image: experiment4 output.png



Student Name: THOTAKURA BALA SUSHMA **Date: Aug 25, 2026, 2:39:19 PM**

Experiment-5

AIM:

The aim of this experiment is to:

- Understand the difference between StatelessWidget and StatefulWidget in Flutter.
- Implement local state management using `setState()` for a small UI component.
- Implement shared app state management using the Provider package.
- Build a fully responsive UI that adapts to mobile, tablet, and desktop screen sizes.
- Integrate both state management approaches with a responsive layout.

PROBLEM STATEMENT:

Create a Flutter application that displays a catalog of items, with the following features:

Responsive Layout

- Mobile (width < 600): Single-column `ListView` of items.
- Tablet (600 "dwidth < 1024): Two-column `GridView`.
- Desktop (width "e1024): `NavigationRail` + three-column `GridView`.

Shared App State (Provider)

- Each item has a favorite state.
- A favorite count badge is displayed in the AppBar (or NavigationRail on desktop).
- Using Provider, tapping the star icon on any item toggles its favorite status.
- The favorite count updates globally across all layouts.

Local State (setState)

- A temporary counter widget exists only in the desktop layout.
- Incrementing the counter does not affect any other layout.
- The counter demonstrates widget-specific local state management.

Write the complete Flutter code (`lib/main.dart`) that satisfies all these requirements.

INPUT:

- No explicit user input.
- The application automatically uses the device's screen dimensions and orientation.
- All catalog data is predefined inside the application model.

OUTPUT:

- A responsive UI that displays Mobile, Tablet, or Desktop layouts based on screen width.
- Favorite count updates correctly across all layouts using Provider.
- Desktop-only counter increments independently using `setState()`.

Example Visual Description

Mobile

- List of items displayed vertically.
- Each item contains a star icon.
- Tapping the star toggles favorite status.
- AppBar displays the total number of favorites.

Tablet

- Two-column grid of item cards.
- Same favorite toggle functionality.
- Favorite count remains synchronized.

Desktop

- NavigationRail displayed on the left.
- Three-column grid of item cards.
- Favorite badge shown in the NavigationRail.
- Separate counter widget below the grid managed by `setState()`.

ALGORITHM:

Step-1: Create a new Flutter project:

```
flutter create responsive_state_demo
```

Step-2: Add the Provider dependency in `pubspec.yaml`:

```
dependencies:
  provider: ^6.1.2
```

Step-3: Create an `Item` model containing:

- `id`
- `name`
- `isFavorite`

Step-4: Create a `CatalogModel` class extending `ChangeNotifier`.

The model should:

- Maintain a list of items.
- Provide a `toggleFavorite(id)` method.
- Provide a `favoriteCount` getter.
- Call `notifyListeners()` whenever data changes.

Step-5: Wrap the root widget with `ChangeNotifierProvider`.

```
ChangeNotifierProvider(
  create: (_) => CatalogModel(),
  child: MyApp(),
)
```

Step-6: Use `LayoutBuilder` to determine the appropriate layout according to screen width.

Step-7: Create separate layout widgets:

- `MobileLayout`
- `TabletLayout`
- `DesktopLayout`

Step-8: Access application state using:

```
context.watch<CatalogModel>()
```

to rebuild widgets whenever favorites change.

Step-9: Create a reusable `ItemCard` widget.

The card should:

- Display item information.

- Show a favorite icon.
- Toggle favorite status using:

```
Provider.of<CatalogModel>(
  context,
  listen: false,
)
```

to avoid unnecessary rebuilds.

Step-10: Add a stateful inside

The counter should:

- Maintain local state using `setState()`.
- Increment independently from Provider state.

Step-11: Run the application on mobile, tablet, and desktop devices and verify:

- Responsiveness
- Favorite synchronization
- Independent desktop counter behavior

EXAMPLES:

Example: Creating a ChangeNotifier Model

```
class CartModel extends ChangeNotifier {
  List<Item> _items = [];

  int get itemCount => _items.length;

  void addItem(Item item) {
    _items.add(item);
    notifyListeners();
  }

  void removeItem(Item item) {
    _items.remove(item);
    notifyListeners();
  }
}
```

Example: Using context.watch()

```
final catalog = context.watch<CatalogModel>();

return Text(
  'Favorites: ${catalog.favoriteCount}',
);
```

Example: Using LayoutBuilder

```
LayoutBuilder(
  builder: (context, constraints) {
```

```
    if (constraints.maxWidth >= 1024) {
      return DesktopLayout();
    }

    if (constraints.maxWidth >= 600) {
      return TabletLayout();
    }

    return MobileLayout();
  },
);
```

Ø=ÜÝ N O T E S - S T A T E L E S S V S S T A T E F U L W I

1. StatelessWidget vs StatefulWidget

In Flutter, every UI element is a widget. Widgets are categorized based on whether they can change after being built.

StatelessWidget

- Immutable after creation.
- Suitable for static UI elements.
- Does not maintain internal state.

Examples:

- Text
- Icon
- Row
- Column

StatefulWidget

- Mutable and dynamic.
- Maintains state through a separate `State<T>` class.
- Rebuilds whenever `setState()` is called.

Examples:

- Forms
- Counters
- Animations
- Switches

Characteristic	StatelessWidget	StatefulWidget
Mutability	Immutable	Mutable
Performance	Lightweight	Slightly heavier
Use Case	Static UI	Dynamic UI
Lifecycle	build()	initState(), build(), dispose()
Boilerplate	Less	More

Key Insight:
A StatelessWidget can receive new data from its parent, but it cannot own changing data itself. Whenever a widget needs to manage mutable data, use StatefulWidget.

2. setState() - Local State Management

`setState()` is Flutter's built-in mechanism for updating widget-specific state.

Example:

```
class _CounterWidgetState extends State<CounterWidget> {
  int _counter = 0;

  void _increment() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $_counter'),
        ElevatedButton(
          onPressed: _increment,
          child: Text('Increment'),
        ),
      ],
    );
  }
}
```

Best Practices

- Keep state local whenever possible.
- Minimize rebuild scope.
- Check `mounted` before calling `setState()` after async operations.
- Never call `setState()` inside `build()`.

Suitable For

- Counters
- Checkboxes
- Toggles
- Temporary UI states

3. Provider - Shared App State Management

Provider is a lightweight state-management solution built on top of `InheritedWidget`.

Benefits

- Avoids callback chains.
- Supports scoped rebuilds.
- Easy to maintain and test.
- Officially recommended for beginners.

Core Concepts

ChangeNotifier

Stores mutable data and notifies listeners.

```
class CounterModel extends ChangeNotifier {
  int _count = 0;

  int get count => _count;
```

```
  void increment() {
    _count++;
    notifyListeners();
  }
}
```

ChangeNotifierProvider

Provides state to descendant widgets.

```
ChangeNotifierProvider(
  create: (context) => CounterModel(),
  child: MyApp(),
);
```

Reading State

```
context.watch<CounterModel>().count;
```

Updating State

```
context.read<CounterModel>().increment();
```

Provider vs setState

Aspect	setState	Provider
Scope	Local	Global
Boilerplate	Minimal	Moderate
Rebuilds	Widget subtree	Only listeners
Best For	Counters, forms	Shared app data

4. Responsive UI in Flutter

Flutter provides multiple tools for responsive design.

MediaQuery

Used for obtaining:

- Screen width
- Screen height
- Orientation
- Safe area information

Example:

```
double width = MediaQuery.of(context).size.width;
```

LayoutBuilder

Provides parent constraints.

Example:

```
LayoutBuilder(
  builder: (context, constraints) {
    return Text(
      '${constraints.maxWidth}',
    );
  },
);
```

OrientationBuilder

Rebuilds widgets when device orientation changes.

Example:

```
OrientationBuilder(
  builder: (context, orientation) {
    return orientation == Orientation.portrait
        ? PortraitLayout()
        : LandscapeLayout();
  },
);
```

Common Breakpoints

Device Category	Width Range	Recommended Layout
Mobile	< 600	Single-column
Tablet	600-1024	Two-column grid
Desktop	"e1024	NavigationRail + three-column grid

Example Layout Switching

```
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth >= 1024) {
      return DesktopLayout();
    }

    if (constraints.maxWidth >= 600) {
      return TabletLayout();
    }

    return MobileLayout();
  },
);
```

Combining State Management and Responsive Design

- Store shared data in Provider.
- Use LayoutBuilder for screen-size decisions.
- Use `setState()` only for local UI interactions.
- Keep layout logic independent from business data.

CONSTRAINTS:

- The application must run without errors on Android, iOS, and Web/Desktop.
- Use Provider for shared application state.
- Use `setState()` only for local widget-specific state.
- Avoid hard-coded widths that may cause overflow.
- Favorite count must remain synchronized across all layouts.
- Responsive behavior must work correctly during screen resizing and orientation changes.

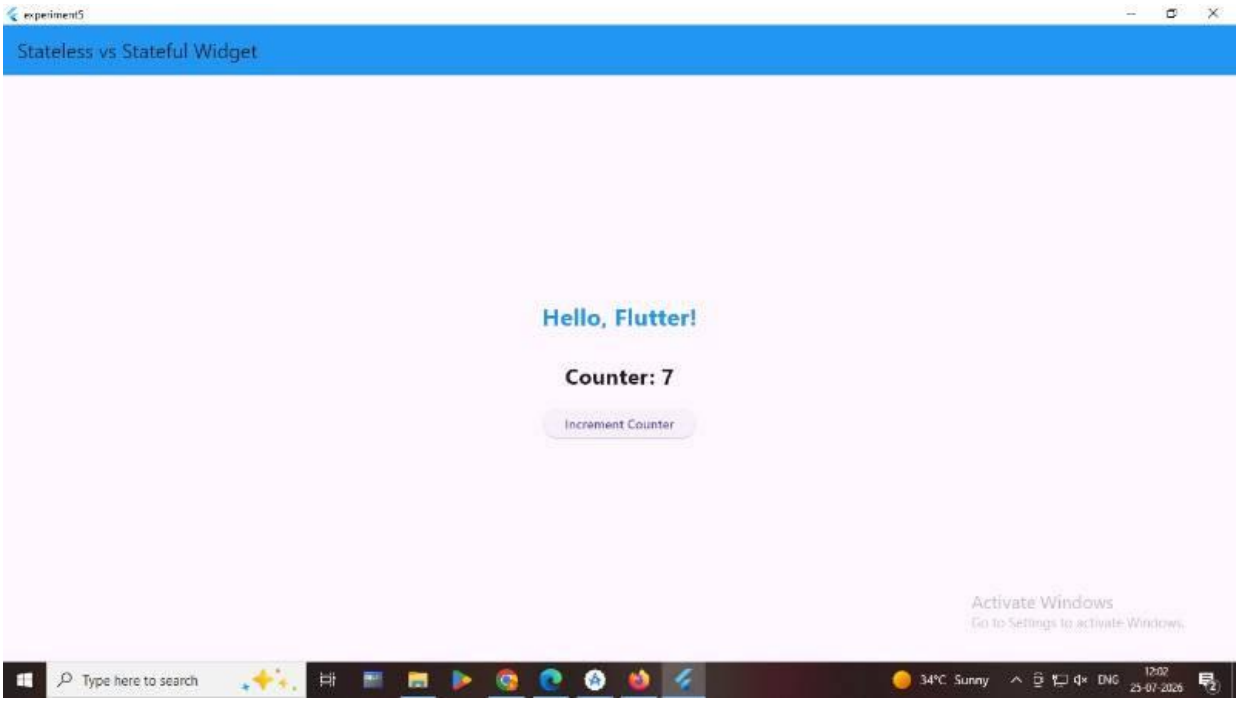
STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutter-exp---5.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutter-exp---5
Default Branch: main
Fetched Files:
```

Output Image: experiment-5 output.png



Student Name: THOTAKURA BALA SUSHMA **Date: Sep 2, 2026, 11:49:07 AM**

Experiment-6

AIM:

The aim of this experiment is to:

(a) Create custom, reusable widgets for specific UI elements (e.g., a custom card, a custom button, and a custom avatar) to demonstrate encapsulation and reusability.

(b) Apply styling using Flutter's theming system by defining a global `ThemeData` and custom styles that can be overridden locally.

PROBLEM STATEMENT:

Build a Flutter application that displays a profile screen with the following:

1. A **CustomAvatar** widget that shows a circular avatar with an image or initials.
   - Accepts parameters: `radius`, `imageUrl`, and `backgroundColor`.
2. A **ProfileCard** widget that displays:
   - Title
   - Subtitle
   - Optional trailing icon
   - Tap callback
3. A **CustomButton** widget that:
   - Uses rounded corners
   - Uses a gradient background
   - Accepts `label` and `onPressed` parameters
4. Define a global theme using `ThemeData`:
   - Primary color: **Teal**
   - Secondary color: **Amber**
   - Custom text theme for headline and body text
   - Custom elevated button style with rounded corners and padding
5. Use all custom widgets on a profile screen and demonstrate how theme changes affect all widget instances.

INPUT:

- No explicit user input.
- The UI content is static.

OUTPUT:

A profile screen containing:

- A circular avatar (custom widget)
- A profile card displaying name and email
- An **Edit Profile** button
- A **Settings** button using the same custom widget with different styling
- Consistent styling applied through Flutter's theme system

Example Visual Description

- The avatar appears as a circle with an image or colored placeholder.
- The profile card has rounded corners, elevation, and a trailing settings icon.
- Buttons have rounded corners, gradient backgrounds, and white text.
- Theme colors are applied consistently throughout the application.

ALGORITHM:

Step-1: Create a new Flutter project:

```
flutter create custom_widgets_theming
```

Step-2: Open `lib/main.dart` and define a custom `ThemeData` inside `MaterialApp`.

Step-3: Create reusable custom widgets:

- `CustomAvatar`
- `ProfileCard`
- `CustomButton`

Step-4: Implement `CustomAvatar` using `CircleAvatar` with support for image URLs and initials fallback.

Step-5: Implement `ProfileCard` using `Card` and `ListTile`.

Step-6: Implement `CustomButton` using `ElevatedButton` and apply custom styles using the app theme.

Step-7: Create a `ProfileScreen` widget and assemble all custom widgets.

Step-8: Run the application and verify that:

- Theme colors are applied correctly.
- Widgets are reusable.
- Local style overrides work when needed.

EXAMPLES:

EXAMPLE: CUSTOMAVATAR WIDGET:

```
class CustomAvatar extends StatelessWidget {
  final double radius;
  final String? imageUrl;
  final String? initials;
  final Color backgroundColor;

  const CustomAvatar({
    required this.radius,
    this.imageUrl,
    this.initials,
    this.backgroundColor = Colors.grey,
  });

  @override
  Widget build(BuildContext context) {
    return CircleAvatar(
      radius: radius,
      backgroundColor: backgroundColor,
      backgroundImage:
          imageUrl != null ? NetworkImage(imageUrl!) : null,
      child: imageUrl == null && initials != null
          ? Text(
              initials!,
              style: TextStyle(
                fontSize: radius * 0.8,
              ),
            )
          : null,
    );
  }
}
```

CONSTRAINTS:

- All custom widgets must be reusable and configurable through parameters.
- Avoid hard-coded values whenever possible.
- Define the global theme inside `MaterialApp`.
- Use `Theme.of(context)` to access theme properties.
- Avoid duplicating styling code across widgets.
- Ensure the application runs correctly on Android, iOS, and Web platforms.

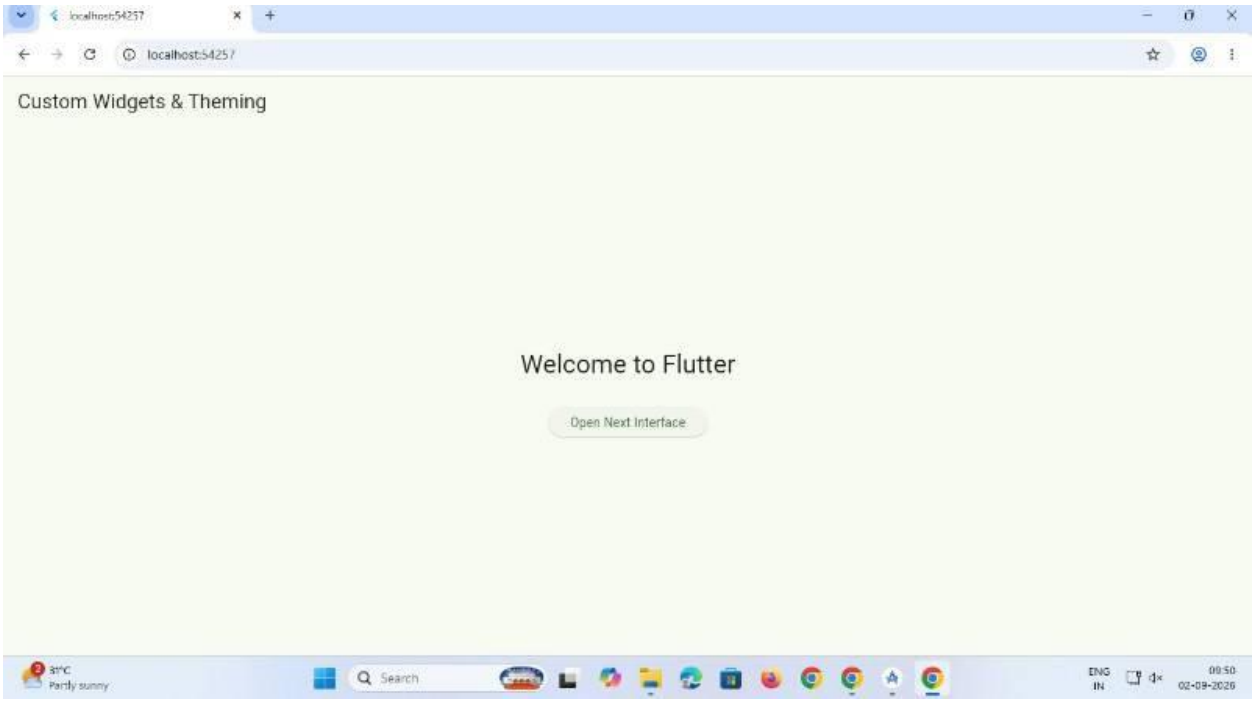
STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutterlab_exp---6.git
Notes:
```

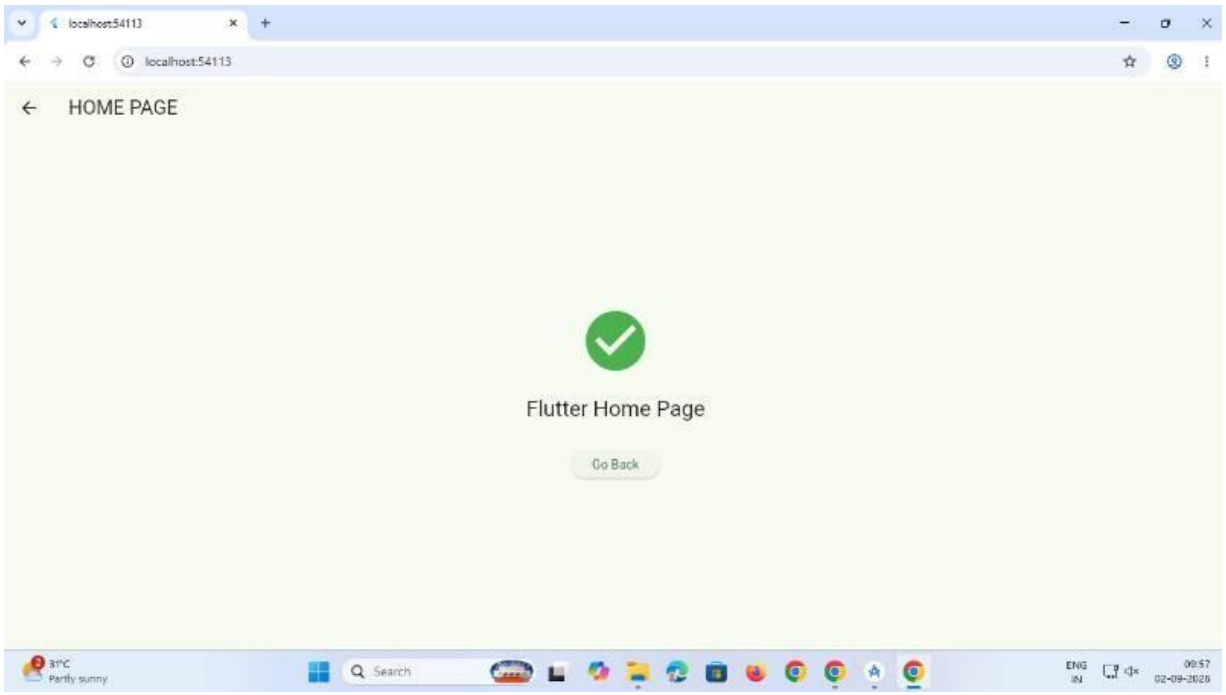
OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutterlab_exp---6
Default Branch: main
Fetched Files:
```

Output Image: exp6 output-1.jpeg



Output Image: exp6 output-2.png



Student Name: THOTAKURA BALA SUSHMA **Date: Oct 5, 2026, 11:01:54 PM**

Experiment-7

AIM:

The aim of this experiment is to:

(a) Design a form with various input fields (name, email, password, confirm password, phone number, age, gender dropdown, and date of birth).

(b) Implement form validation and error handling by displaying appropriate error messages when fields are empty or contain invalid data, and showing a success message when the form is submitted correctly.

PROBLEM STATEMENT:

Create a Flutter application that displays a **Registration Form** with the following fields:

1. **Full Name** (Text Input)
   - Required
   - Minimum 3 characters
2. **Email** (Email Input)
   - Required
   - Must be a valid email format
3. **Password** (Password Input)
   - Required
   - Minimum 6 characters
4. **Confirm Password**
   - Required
   - Must match the Password field
5. **Phone Number**
   - Required
   - Numeric only
   - Minimum 10 digits
6. **Age**
   - Required
   - Must be between 1 and 120
7. **Gender**
   - Dropdown (Male, Female, Other)
   - Required
8. **Date of Birth**
   - Date Picker
   - Required
   - Future dates are not allowed

The form must:

- Show inline error messages below each field when validation fails.
- Prevent submission if any field contains invalid data.
- Display a success `SnackBar` (or dialog) when all fields are valid.
- Optionally clear the form after successful submission.
- Provide a **Reset** button that clears all fields and validation messages.

INPUT:

- User enters values into text fields.
- User selects a gender from the dropdown.
- User selects a date using the date picker.

OUTPUT:

Success Output

```
SnackBar: Registration Successful!
```

Example Validation Errors

```
Please enter your name
```

```
Enter a valid email address
```

```
Passwords do not match
```

```
Phone number must contain at least 10 digits
```

ALGORITHM:

Step-1: Create a new Flutter project:

```
flutter create form_validation_demo
```

Step-2: Open `lib/main.dart` and create a `StatefulWidget` to manage form data.

Step-3: Create a `GlobalKey<FormState>` for form validation.

Step-4: Create `TextEditingController` objects for:

- Name
- Email
- Password
- Confirm Password
- Phone Number
- Age
- Date of Birth

Step-5: Add a `Form` widget containing multiple `TextFormField` widgets.

Step-6: Implement custom validator functions for each field.

Step-7: Add a `DropdownButtonFormField` for Gender selection with validation.

Step-8: Integrate a date picker using `showDatePicker()`.

Step-9: Add a **Submit** button:

```
if (_formKey.currentState!.validate()) {
   // Show success message
}
```

Step-10: Add a **Reset** button that:

- Clears all text controllers
- Resets dropdown selection
- Clears date selection
- Removes validation messages

Step-11: Run the application and test all validation scenarios.

EXAMPLES:

EXAMPLE: EMAIL VALIDATOR:

```
String? validateEmail(String? value) {
  if (value == null || value.isEmpty) {
    return 'Email is required';
  }

  if (!value.contains('@') || !value.contains('.')) {
    return 'Enter a valid email';
  }

  return null;
}
```

EXAMPLE: PASSWORD VALIDATOR:

```
String? validatePassword(String? value) {
  if (value == null || value.isEmpty) {
    return 'Password is required';
  }

  if (value.length < 6) {
    return 'Password must be at least 6 characters';
  }

  return null;
}
```

EXAMPLE: CONFIRM PASSWORD VALIDATOR:

```
String? validateConfirmPassword(String? value) {
  if (value == null || value.isEmpty) {
    return 'Confirm your password';
  }

  if (value != _passwordController.text) {
    return 'Passwords do not match';
  }

  return null;
}
```

EXAMPLE: DATE PICKER INTEGRATION:

```
TextFormField(
  controller: _dobController,
  readOnly: true,
  onTap: () async {
    DateTime? picked = await showDatePicker(
      context: context,
      initialDate: DateTime.now(),
      firstDate: DateTime(1900),
      lastDate: DateTime.now(),
    );

    if (picked != null) {
      _dobController.text =
          picked.toLocal().toString().split(' ')[0];
    }
  },
  validator: (value) =>
      value == null || value.isEmpty
          ? 'Date of Birth is required'
          : null,
)
```

CONSTRAINTS:

- All fields must be validated before form submission.
- Inline error messages must appear below the corresponding fields.
- The Date Picker must restrict users from selecting future dates.
- Do not use third-party validation libraries.
- Implement all validation logic using custom validator functions.
- The Reset button must clear all fields and remove validation errors.
- The application should run correctly on Android, iOS, and Web platforms.
- Ensure proper user experience with meaningful validation messages.

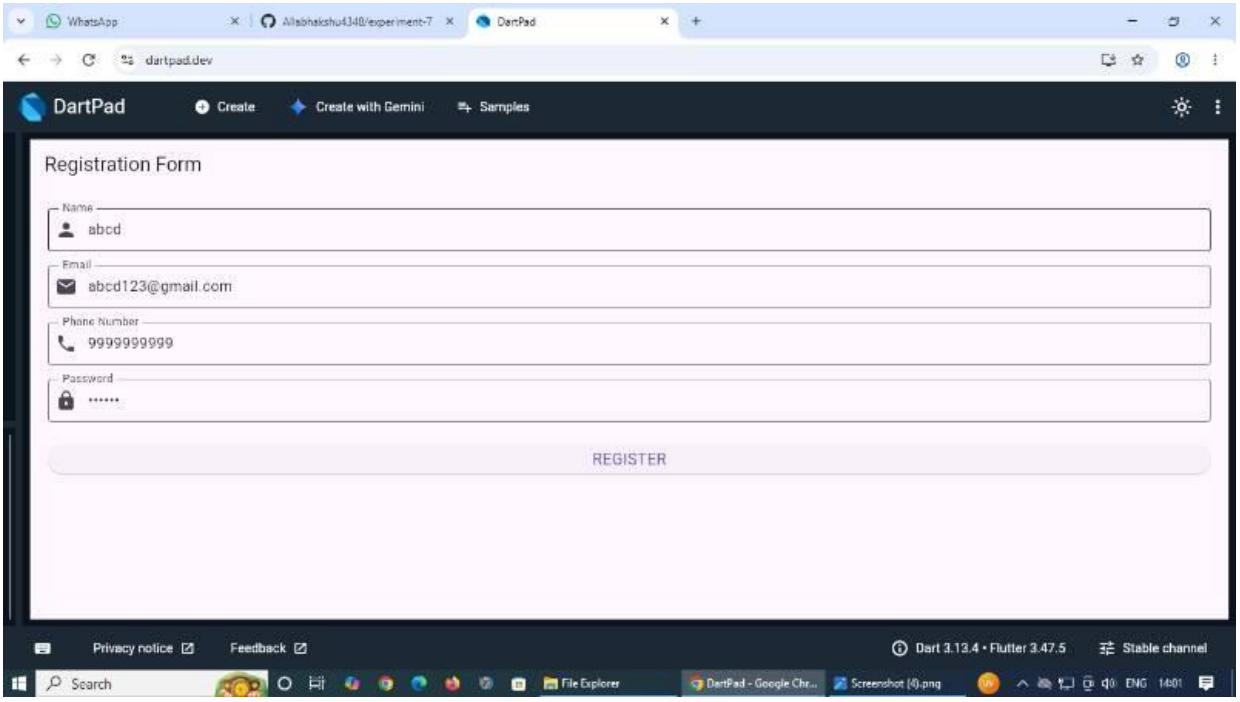
STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutter-exp-7.git
Notes:
```

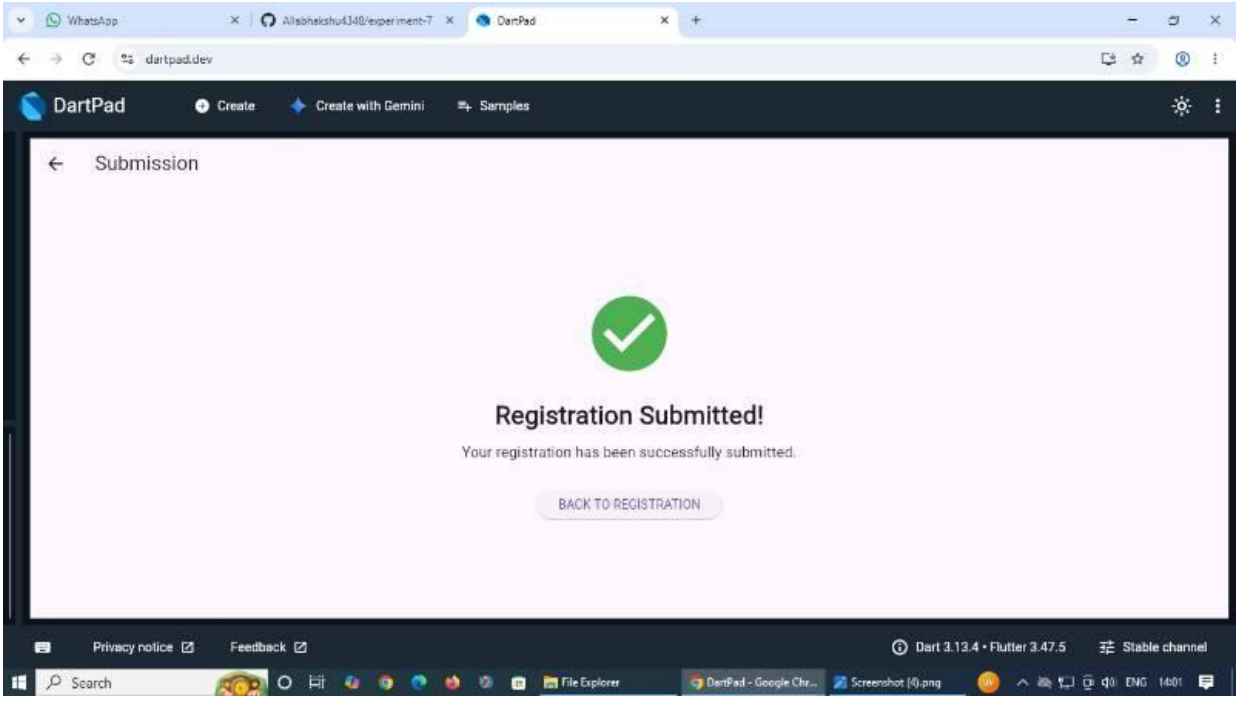
OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutter-exp-7
Default Branch: main
Fetched Files:
```

Output Image: output7(1).png



Output Image: output7(2).png



Student Name: THOTAKURA BALA SUSHMA **Date: Oct 1, 2026, 11:40:17 PM**

Experiment-8

AIM:

The aim of this experiment is to:

(a) Add animations to UI elements using Flutter's animation framework.

(b) Experiment with different types of animations (Fade, Slide, Scale, Rotation) and combine them to create smooth and engaging user experiences.

PROBLEM STATEMENT:

Create a Flutter application that demonstrates the following animation types using a card-based UI:

1. Fade Animation
- A card fades in when the application loads.
- A button allows the user to fade the card out.

2. Slide Animation
- A card slides into the screen from the left or right when a button is pressed.

3. Scale Animation
- A card scales up with a bounce effect when tapped.

4. Rotation Animation
- A card rotates 360° when a button is pressed.

5. Combined Animation
- A card simultaneously fades and slides into view.

The application should provide controls to trigger each animation individually and include a reset option that restores all cards to their original state.

INPUT:

- User taps buttons to trigger animations.
- User taps animated cards for interactive effects.

OUTPUT:

Visual animations displayed on screen based on user interaction.

Example Visual Description

- First card fades in when the application starts and fades out on button press.
- Second card slides in from the left side.
- Third card enlarges with a bounce effect when tapped.
- Fourth card rotates completely (360°) when a button is pressed.
- Fifth card combines fade and slide animations simultaneously.

ALGORITHM:

Step-1: Create a new Flutter project:

```
flutter create animations_demo
```

Step-2: Open `lib/main.dart` and create a `StatefulWidget`.

Step-3: Use `SingleTickerProviderStateMixin` or `TickerProviderStateMixin` to manage animation controllers.

Step-4: Create separate `AnimationController` instances for:

- Fade Animation
- Slide Animation
- Scale Animation
- Rotation Animation
- Combined Animation

Step-5: Implement Fade Animation using:

```
FadeTransition
```

or

```
AnimatedOpacity
```

Step-6: Implement Slide Animation using:

```
SlideTransition
```

with a `Tween<Offset>`.

Step-7: Implement Scale Animation using:

```
ScaleTransition
```

with a `CurvedAnimation`.

Step-8: Implement Rotation Animation using:

```
RotationTransition
```

Step-9: Create buttons that trigger:

```
_controller.forward();
```

and

```
_controller.reverse();
```

Step-10: Create a combined Fade + Slide animation using:

```
AnimatedBuilder
```

or multiple transition widgets.

Step-11: Add a Reset button that returns all animations to their initial state.

Step-12: Run the application and test all animations.

EXAMPLES:

EXAMPLE: SLIDETRANSITION:

```
SlideTransition(
  position: Tween<Offset>(
    begin: const Offset(-1, 0),
    end: Offset.zero,
  ).animate(_slideController),
  child: Card(
    child: Padding(
      padding: EdgeInsets.all(16),
      child: Text('Sliding Card'),
    ),
  ),
)
```

EXAMPLE: SCALETRANSITION:

```
final scaleAnimation = Tween<double>(
  begin: 1.0,
  end: 1.5,
).animate(
  CurvedAnimation(
    parent: _scaleController,
    curve: Curves.easeOutBack,
  ),
);

ScaleTransition(
  scale: scaleAnimation,
  child: Card(
    child: Padding(
      padding: EdgeInsets.all(16),
      child: Text('Scaling Card'),
    ),
  ),
)
```

EXAMPLE: ROTATIONTRANSITION:

```
RotationTransition(
  turns: _rotationController,
  child: Card(
    child: Padding(
      padding: EdgeInsets.all(16),
      child: Text('Rotating Card'),
    ),
  ),
)
```

EXAMPLE: FADETRANSITION:

```
FadeTransition(
  opacity: _fadeAnimation,
  child: Card(
    child: Padding(
      padding: EdgeInsets.all(16),
      child: Text('Fade Card'),
    ),
  ),
)
```

EXAMPLE: COMBINED FADE + SLIDE ANIMATION:

```
SlideTransition(
  position: _slideAnimation,
  child: FadeTransition(
    opacity: _fadeAnimation,
    child: Card(
      child: Padding(
        padding: EdgeInsets.all(16),
        child: Text('Combined Animation Card'),
      ),
    ),
  ),
)
```

CONSTRAINTS:

- Use both implicit and explicit animations where appropriate.
- Dispose all `AnimationController` objects inside the `dispose()` method.
- Avoid layout overflow during animation.
- Ensure smooth transitions and proper performance.
- Animation widgets should remain responsive during repeated interactions.
- Use Flutter's built-in animation framework only.
- The application must run correctly on Android, iOS, and Web platforms.
- Reset functionality should restore all animated widgets to their initial state.

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutter-exp-8.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutter-exp-8
Default Branch: main
Fetched Files:
```

Student Name: THOTAKURA BALA SUSHMA **Date: Oct 2, 2026, 10:46:23 PM**

Experiment-9

AIM:

The aim of this experiment is to:

(a) Fetch data from a REST API using the `http` package in Flutter.
(b) Display the fetched data in a meaningful way in the UI, with proper loading indicators and error handling.

PROBLEM STATEMENT:

Create a Flutter application that:

- Fetches a list of posts from the public API: `https://jsonplaceholder.typicode.com/posts`
- Displays the posts in a list showing the title and a preview of the body.
- When a post is tapped, navigate to a detail screen showing the full title and body.
- Shows a loading indicator while fetching data.
- Shows an error message with a retry button if fetching fails.
- Uses a proper model class and JSON parsing.

INPUT:

- No explicit user input.
- The application automatically fetches data from the API when launched.

OUTPUT:

Main Screen
- Displays a loading indicator while data is being fetched.
- Displays a list of posts after successful retrieval.
- Displays an error message with a retry option if the request fails.

Detail Screen
- Displays the complete title and body of the selected post.

Example Visual Output

Screen 1
- CircularProgressIndicator while loading.
- List of posts after loading completes.
- Each list item displays:
  - Post title (bold)
  - One-line body preview

Screen 2
- AppBar titled **"Post Details"**
- Full post title
- Full post body

ALGORITHM:

Step-1
Add the `http` package to `pubspec.yaml`.

Step-2
Create a `Post` model class with a `fromJson()` constructor.

Step-3
Create a `PostService` class with a `fetchPosts()` method that performs an HTTP GET request.

Step-4
Use `FutureBuilder<List<Post>>` to asynchronously fetch data from the API.

Step-5
Inside `FutureBuilder`:

- If `connectionState == ConnectionState.waiting`
  - Show `CircularProgressIndicator`.
- If an error occurs
  - Show an error message.
  - Display a Retry button.
- If data is available
  - Display posts using `ListView.builder`.

Step-6
When a post is tapped:

- Navigate to a `DetailScreen`.
- Pass the selected `Post` object.

Step-7
Display the complete title and body on the detail screen.

Step-8
Test the application with:

- Internet connection enabled.
- Internet connection disabled.

Verify proper loading and error handling.

EXAMPLES:

Example: fetchPosts() Function

```
Future<List<Post>> fetchPosts() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  );

  if (response.statusCode == 200) {
    List jsonList = jsonDecode(response.body);

    return jsonList
        .map((json) => Post.fromJson(json))
        .toList();
  } else {
    throw Exception('Failed to load posts');
  }
}
```

CONSTRAINTS:

- Must use `FutureBuilder` or a `StatefulWidget` with manual state management.
- Must handle:
  - Loading state
  - Error state
  - Success/Data state

- Do not hard-code post data.
- Fetch data from the API each time the application starts.
- Ensure the application does not crash during network failures.
- Use proper JSON parsing through model classes.
- Implement navigation to a detail screen for viewing complete post information.

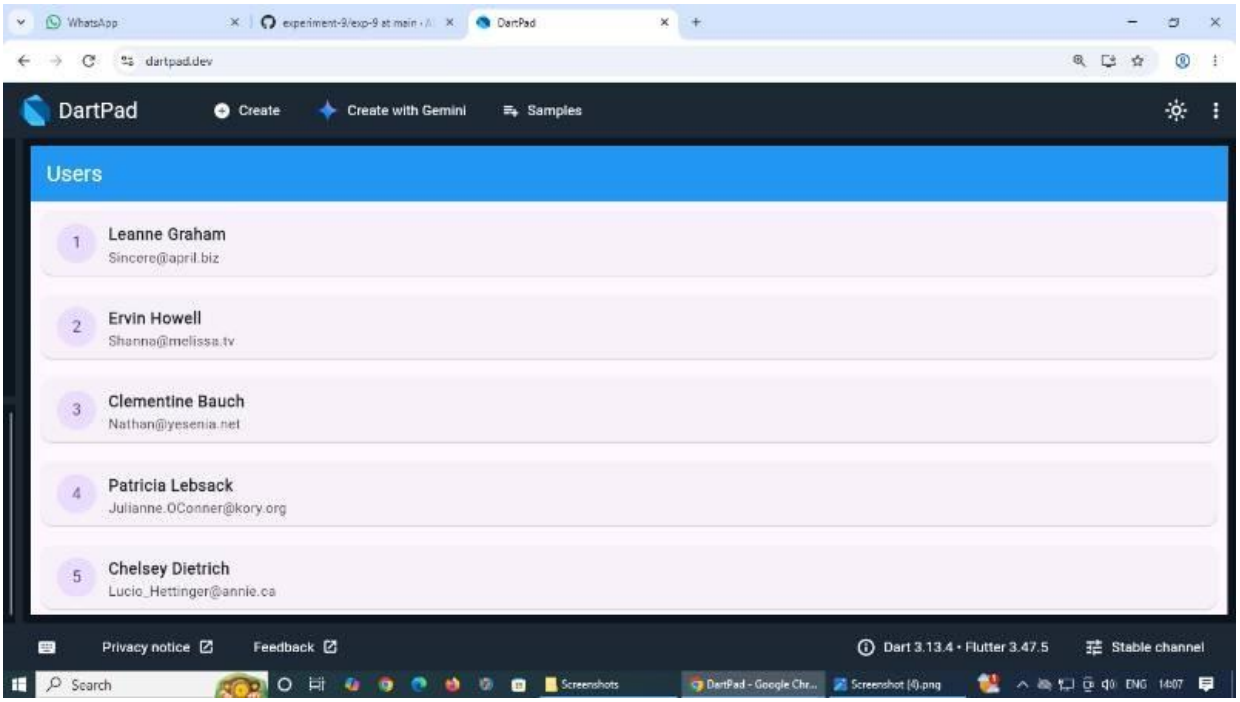
STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutter-exp-9.git
Notes:
```

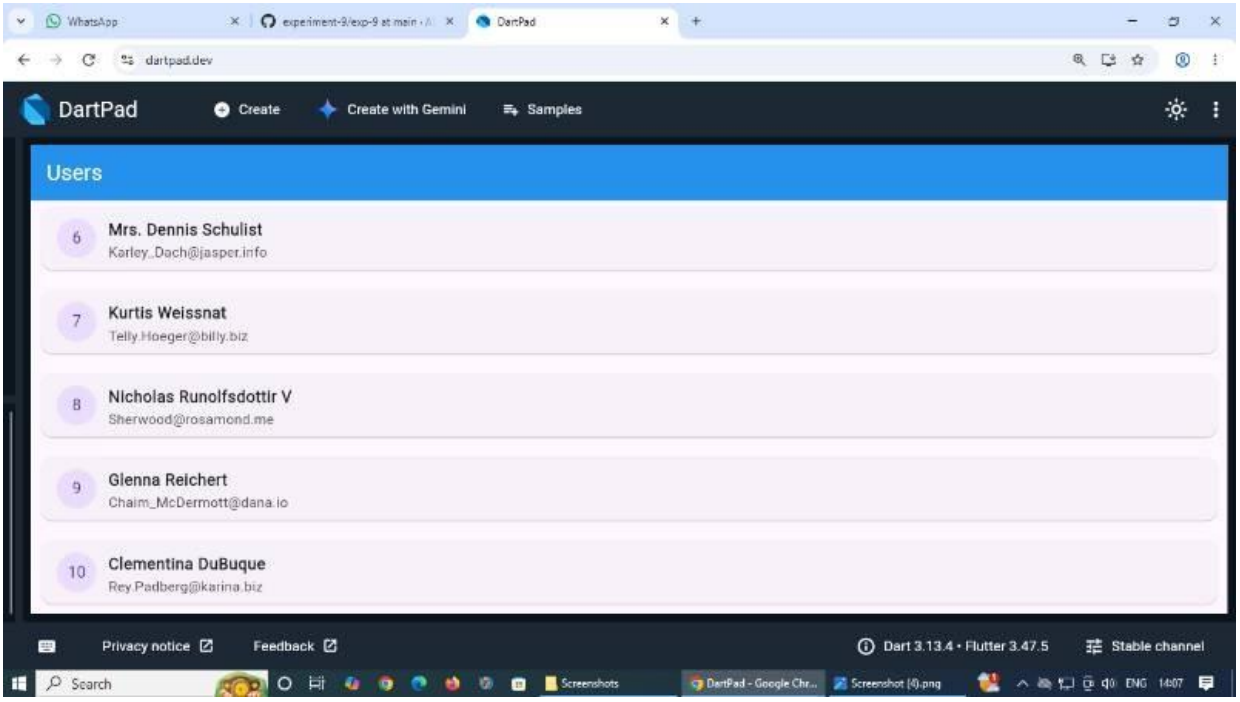
OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutter-exp-9
Default Branch: main
Fetched Files:
```

Output Image: output9(1).png



Output Image: output9(2).png



Student Name: THOTAKURA BALA SUSHMA **Date: Oct 5, 2026, 11:14:51 PM**

Experiment-10

AIM:

The aim of this experiment is to:

(a) Write unit tests for UI components to verify their behaviour (e.g., button tap updates a counter, input validation works).

(b) Use Flutter's debugging tools (console, breakpoints, inspector) to identify and fix issues in a sample buggy app.

PROBLEM STATEMENT:

You are given a buggy counter application that contains two issues:

1. The counter does not update when the increment button is tapped (state management bug).
2. When the counter reaches **10**, the counter text overflows its container (layout overflow issue).

Tasks

Part (a): Write Unit Tests

Create widget tests for the corrected counter application to verify:

- Initial counter value is **0**.
- Tapping the increment button increases the counter by **1**.
- Tapping the decrement button decreases the counter by **1**.
- Counter value never goes below **0**.
- Counter text is displayed correctly on the screen.

Part (b): Debug and Fix the Application

Use Flutter debugging tools to:

- Identify the state management issue.
- Identify the layout overflow issue.
- Fix both problems.
- Document the debugging process.

INPUT:

- No direct user input.
- The buggy Flutter application source code.
- Widget test code.

OUTPUT:

Corrected Application

- Counter increments properly.
- Counter decrements properly.
- Counter never becomes negative.
- No layout overflow occurs when the counter reaches 10 or higher.

Test Results

- All widget tests pass successfully when running:

```
flutter test
```

Debugging Log

A documented explanation showing:

- How each bug was discovered.
- Which debugging tool was used.
- How the bug was fixed.

Example Debugging Log

```
Bug 1: Counter not incrementing.
- Added print() inside onPressed callback.
- No output appeared in console.
- Checked button callback assignment.
- Found onPressed was null.
- Fixed by assigning:
  onPressed: _incrementCounter

Bug 2: Overflow when counter reaches 10.
- Used Flutter Inspector.
- Observed yellow/red overflow indicators.
- Enabled debugPaintSizeEnabled.
- Found Text widget exceeded available width.
- Fixed by wrapping Text with Expanded/Flexible and constraining layout.
```

ALGORITHM:

PART (A) - WRITING WIDGET TESTS:

Step-1
Create a Flutter project or use the provided project.

Step-2
Navigate to the `test` folder.

Step-3
Create a test file:

```
test/widget_test.dart
```

Step-4
Import Flutter testing libraries and the application widget.

Step-5
Write a widget test to verify the initial counter value.

Step-6
Write a widget test to verify increment functionality.

Step-7
Write a widget test to verify decrement functionality.

Step-8
Ensure the counter cannot go below zero.

Step-9
Verify that the counter text is rendered correctly.

Step-10
Run all tests using:

```
flutter test
```

PART (B) - DEBUGGING THE APPLICATION:

Step-1
Run the buggy application on an emulator or physical device.

Step-2
Tap the increment button and observe that the counter does not update.

Step-3
Add a debug statement inside the callback:

```
print('increment called');
```

Step-4
Check whether the message appears in the console.

Step-5
Inspect the button's `onPressed` assignment and locate the issue.

Step-6
Fix the callback so that the increment function executes correctly.

Step-7
Increase the counter value until it reaches 10.

Step-8
Observe the overflow warning shown by Flutter.

Step-9
Use Flutter Inspector or `debugPaintSizeEnabled` to inspect layout boundaries.

Step-10
Identify the widget causing the overflow.

Step-11
Fix the layout using widgets such as:

- `Expanded`
- `Flexible`
- `FittedBox`
- Proper width constraints

Step-12
Re-run the application and verify the overflow issue is resolved.

Step-13
Execute widget tests again and ensure all tests pass.

EXAMPLES:

EXAMPLE 1 - WIDGET TEST FOR INCREMENT BUTTON:

```
testWidgets('Counter increments when button tapped', (tester) async {
  await tester.pumpWidget(MyApp());
```

```
  expect(find.text('0'), findsOneWidget);

  await tester.tap(find.byIcon(Icons.add));
  await tester.pump();

  expect(find.text('1'), findsOneWidget);
});
```

EXAMPLE 2 - WIDGET TEST FOR DECREMENT PROTECTION:

```
testWidgets('Counter does not go below zero', (tester) async {
  await tester.pumpWidget(MyApp());

  expect(find.text('0'), findsOneWidget);

  await tester.tap(find.byIcon(Icons.remove));
  await tester.pump();

  expect(find.text('0'), findsOneWidget);
});
```

EXAMPLE 3 - DEBUG PRINT STATEMENT:

```
void _incrementCounter() {
  print('increment called');

  setState(() {
    _counter++;
  });
}
```

EXAMPLE 4 - PREVENTING OVERFLOW:

```
Expanded(
  child: Text(
    'Counter: $_counter',
    overflow: TextOverflow.ellipsis,
  ),
)
```

CONSTRAINTS:

- The buggy application must be fixed before testing.
- All widget tests must pass successfully.
- Tests must not depend on:
  - Network calls
  - APIs
  - External services

- Debugging steps must be documented clearly.
- Use Flutter debugging tools such as:
  - Console logs (`print`)
  - Breakpoints
  - Flutter Inspector
  - `debugPaintSizeEnabled`
- Counter value must never become negative.
- Layout must remain responsive and free from overflow errors.
- The application should behave consistently across different screen sizes.

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/balasushma-nec/flutter-exp-10.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: balasushma-nec
Repo Name: flutter-exp-10
Default Branch: main
Fetched Files:
```